UNIT II
OBJECT, CLASS INTERFACES AND PACKAGES

Object – class – constructor – benefits of OOPS – concepts of OOPS – inheritance – polymorphism – abstract class – generic class – Overriding – Overloading - Interface: Implementation of interface – extending interface – inner class – static and dynamic binding – package: Package as Access Protection – CLASSPATH setting – Import packages.

2.1 Object Oriented Programming

Object-oriented programming is a way of solving a complex problem by breaking them into a small sub-problem. An object is a real-world entity. It is easier to develop a program by using an object. In OOPs, we create programs using class and object in a structured manner.

2.1.1 OOP Concepts:

- Objects
- Classes
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation
- Message Passing

1. Objects:

An object is a real-world entity that can be identified distinctly.

All the objects have a state and a behavior. If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging the tail, running.

If you compare the software object with a real-world object, they have very similar characteristics.

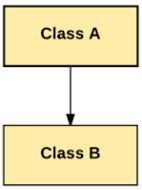
Software objects also have a state and a behavior. A software object's state is stored in fields (attributes, data member) and behavior is shown via methods (member function).

2. Classes:

A class is a template or blueprint or prototype that defines data members and methods of an object. An object is the instance of the class. We can define a class by using the class keyword.

3. Inheritance:

Inheritance is the process of deriving new classes from existing class. Inheritance can be defined as the process where one (parent/super) class acquires the properties (methods and fields) of another (child/sub).



4. Polymorphism:

Polymorphism is the ability of an object to perform different actions (or, exhibit different behaviors) based on the context.

Types of Polymorphism

1. Runtime Polymorphism
2. Compile time Polymorphism

5. Abstraction:

Abstraction is a process of hiding the implementation details from the user, only the functionality will be provided to the user. In other words, the user will have the information on what the object does instead of how it does it.

In Java, abstraction is achieved using Abstract classes and interfaces.

6. Encapsulation:

In encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of own class in which they are declared

7. Message Passing

Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates

the desired results. Message passing involves specifying the name of the object, the name of the function and the information to be sent

Difference between Procedure Oriented Programming and Object Oriented Programming

PROCEDURAL ORIENTED PROGRAMMING	OBJECT ORIENTED PROGRAMMING
In procedural programming, program is divided into small parts called ***functions***.	In object oriented programming, program is divided into small parts called ***objects***.
Procedural programming follows ***top down approach***.	Object oriented programming follows ***bottom up approach***.
There is no access specifier in procedural programming.	Object oriented programming have access specifiers like private, public, protected etc.
Procedural programming does not have any proper way for hiding data so it is ***less secure***.	Object oriented programming provides data hiding so it is ***more secure***.
In procedural programming, overloading is not possible.	Overloading is possible in object oriented programming.
In procedural programming, function is more important than data.	In object oriented programming, data is more important than function.
Procedural programming is based on ***unreal world***.	Object oriented programming is based on ***real world***.
Examples: C, FORTRAN, Pascal, Basic etc.	Examples: C++, Java, Python, C# etc.

2.1.2 Benefits of Object Oriented Programming

1. Modularity

- Code is divided into classes and objects, each with specific responsibilities.
- Makes code organized and easier to manage.

Example: Each real-world entity (e.g., Car, Engine, Wheel) is represented as a class.

2. Reusability (Inheritance)

- You can create new classes from existing ones.
- Promotes code reuse, saving time and effort.

Example: ElectricCar can inherit from the Car class and add specific features.

3. Flexibility and Maintainability (Polymorphism)

- Objects can take many forms, allowing methods to work with different types of objects.
- Easier to update or extend the program without breaking existing code.

Example: You can write one method that works for Dog, Cat, and Bird (all subclasses of Animal).

4. Encapsulation (Data Hiding)

- Data and methods are bundled together.
- Sensitive data is hidden using access modifiers (private, protected, public).

Example: getSalary() method allows access to salary info, but the actual salary variable is private.

5. Improved Debugging and Testing

- Because classes are modular, it's easier to test individual components.
- Bugs can be found and fixed quickly in isolated modules.

6. Scalability

- OOP systems are easier to scale by adding new classes and features without changing existing ones.

7. Real-world Modeling

- OOP mimics real-world entities, making it intuitive and easier for developers to understand and design systems.

8. Security

- Through encapsulation and access control, data can be protected from unauthorized access

2.2 Constructors

- It is a special type of method which is used to initialize the object.
- Constructor name must be the same as its class name
- A Constructor must have no explicit return type
- A Java constructor cannot be abstract, static, final, and synchronized.
- It is called when an instance of the class (object) is created.
- Constructor is called when the object is created for the class. At the time of calling constructor, memory for the object is allocated.

2.2.1 Types of Java constructors

There are two types of constructors in Java:

1. **Default constructor (no-arg constructor)**

```
class Student{
int rollno;
String name;
Student(){
rollno=111;
name=Karan;
}
void displayInformation(){
System.out.println(rollno+" "+name);
}  }
class TestStudent3{
public static void main(String args[]){
Student s1=new Student();
s1.displayInformation();
}
}
```

Output:

```
111 Karan
```

2. **Parameterized constructor**

```
class Student{
int rollno;
String name;
Student(int r, String n){
rollno=r;
name=n;
}
void displayInformation(){
System.out.println(rollno+" "+name);
}
}
class TestStudent3{
public static void main(String args[]){
Student s1=new Student(111,"Karan");
Student s2=new Student(222,"Aryan");
s1.displayInformation();
s2.displayInformation();
}
}
```

Output:

```
111 Karan
222 Aryan
```

2.2.2 Constructor Overloading

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

```
class Student{
  int id;
  String name;
  int age;
  //creating two arg constructor
  Student (int i,String n){
  id = i;
  name = n;
  }
  //creating three arg constructor
  Student (int i,Stringn,int a){
  id = i;
  name = n;
  age=a;
  }
  void display(){System.out.println(id+" "+name+" "+age);
   }
  public static void main(String args[]){
  Student s1 = new Student (111,"Karan");
  Student s2 = new Student (222,"Aryan",25);
  s1.display();
  s2.display();
 }
}
```

Output:

```
111 Karan 0
222 Aryan 25
```

2.2.3 Copy Constructor

There is no copy constructor in Java. However, we can copy the values from one object to another.

There are many ways to copy the values of one object into another in Java. They are:

- By constructor
- By assigning the values of one object into another

1. using Java constructor.

```
class Student{
  int id;
  String name;
  //constructor to initialize integer and string
  Student (int i,String n){
  id = i;
  name = n;
  }
  //constructor to initialize another object
  Student (Student s){
  id = s.id;
  name =s.name;
  }
  void display(){
System.out.println(id+" "+name);
}
```

```
    public static void main(String args[]){
    Student s1 = new Student (111,"Karan");
    Student s2 = new Student (s1);
    s1.display();
    s2.display();
   }
}
```

Output:

111 Karan
111 Karan

2. By assigning the values of one object into another

```
class Student{
    int id;
    String name;
    Student (int i,String n){
    id = i;
    name = n;
    }
    Student (){}
    void display(){
System.out.println(id+" "+name);
}
    public static void main(String args[]){
    Student s1 = new Student (111,"Karan");
    Student s2 = new Student ();
    s2.id=s1.id;
    s2.name=s1.name;
    s1.display();
    s2.display();
   }
}
```

Output:

111 Karan
111 Karan

2.3 Inheritance

It is the concept of deriving new classes from existing class. Here the derived new object acquires all the properties and behaviors of the old object.

When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Syntax

```
class Super {
  .....
}
class Sub extends Super {
  .....
}
```

The extends is the keyword used to inherit the properties of a class. The meaning of "extends" is to increase the functionality

Super Classes: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

Sub Classes: Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.

```
class Teacher {
  String designation = "Professor";
  String collegeName = "ABC";
  void does(){
    System.out.println("Teaching");
  }
}
public class PhysicsTeacher extends Teacher{
  String mainSubject = "Physics";
  public static void main(String args[]){
    PhysicsTeacher obj = new PhysicsTeacher();
    System.out.println(obj.collegeName);
    System.out.println(obj.designation);
    System.out.println(obj.mainSubject);
    obj.does();
  }
}
```

Output:

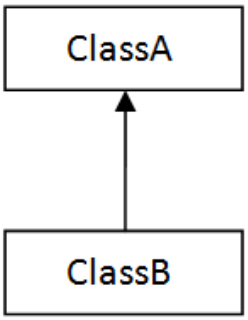
ABC
Professor
Physics
Teaching

Based on the above example we can say that PhysicsTeacher IS-A Teacher. This means that a child class has IS-A relationship with the parent class. This is inheritance is known as IS-A relationship between child and parent class

2.3.1 Types of Inheritance

1. Single Inheritance:

Single class is derived from single base class



Example:

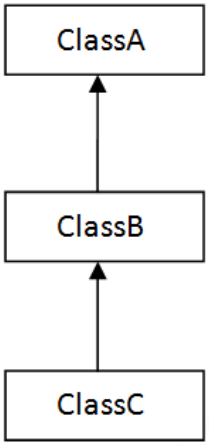
```
class A{
   int a = 10;
   void show() {
System.out.println("a = "+a);
   }
}
public class B extends A{
public static void main(String[] args) {
   B b = new B();
b.show();
   }
}
```

Output

a=10

2. Multilevel Inheritance

A class is derived from another class which itself is a derived class



Example:

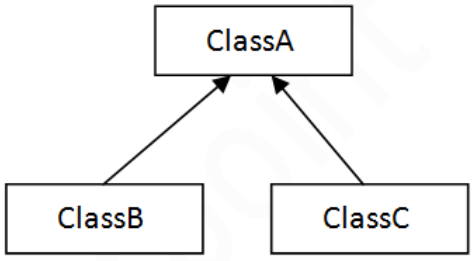
```
class A{
   int a = 10;
   void show() {
System.out.println("a = "+a);
   }
}
class B extends A{
   int b = 10;
   void showB() {
System.out.println("b = "+b);
   }
}
public class C extends B{
public static void main(String[] args) {
   C c = new C();
c.show();
c.showB();
   }
}
```

Output:

a=10
b=10

3. Hierarchical Inheritance

When two or more classes is derived from a single base class, it forms hierarchical inheritance.



Example:

```
class A{
   int a = 10;
   void show() {
System.out.println("a = "+a);
   }
}
class B extends A{
```

```
    int b = 10;
    void showB() {
System.out.println("b = "+b);
    }
}
public class C extends A{
        public static void main(String[] args) {
        C c = new C();
c.show();
        B b = new B();
b.show();
    }
}
```

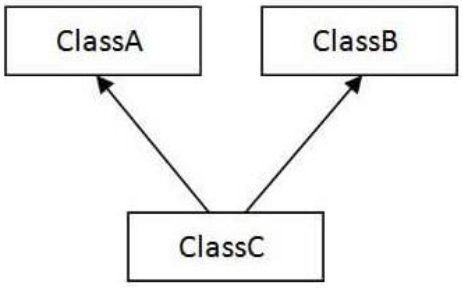
Output:

a = 10

a = 10

4. Multiple Inheritance

A single class is derived from more than one base class



Java does not support multiple inheritance. But it implements the concept of multiple inheritance through interfaces.

Example:

```
class A{
int a = 10;
void show() {
System.out.println("a = "+a);
}
}
interface B{
public void showB();
}
class C extends A implements B{
public void showB() {
int b=10;
System.out.println("b = "+b);
}
public static void main(String args[]){
   C obj=new C();
obj.show();
obj.showB();
}
}
```

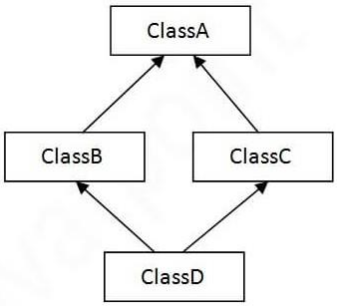
Output:

a = 10

b = 10

5. Hybrid Inheritance:

Hybrid inheritance is a combination of Single and Multiple inheritance.



Example:

```
interface A
{
  public void showA();
}
interface B extends A
{
  public void showB();
}
interface C extends A
{
  public void showC();
}
class D implements B,C
{
 public void showA()
  {
  System.out.println("A");
  }
 public void showB()
  {
  System.out.println("B");
  }
 public void showC()
  {
  System.out.println("C");
  }
  public static void main(String args[]){
  D obj = new D();
  obj.showA();
  obj.showB();
  obj.showC();
  }
}
```

Output:

```
A
B
C
```

2.4 Method Overriding

- Subclass inherits the methods of superclass and sometimes the subclass modifies the implementation of a method defined in superclass
- The method of superclass which gets modified in subclass has the same name and type
- Overridden message must be called from subclass

Program

```
class A
{
int a;
void fun(int i){
this.a=i;
}}
class B extends A{
void fun(int i){
int b=10,c;
super.fun(i+5);
System.out.println("a= "+a);
System.out.println("b= "+b);
c=a*b;
System.out.println("c= "+c);
}
}
class OverrideDemo{
public static void main(String args[]){
B obj=new B();
obj.fun(10);
}}
```

Output->

```
a= 15
b= 10
c= 150
```

Rules for method overriding

- Private data fields in superclass are not accessible to the outside world
- Hence private method cannot be overridden
- Static method can be inherited but cannot be overridden
- Occurs only when the name of the two methods and their type are same

2.5 Method Overloading

Many methods having the same function name but can pass different number of parameters or different types of parameters

Example:

```
int sum(int a,int b);
double sum(double a,double b);
int sum(int a,int b,int c);
```

Example:

```
public class overloadingdemo{
public static void main(String args[]){
sum(10,20);
sum(10.5,20.4);
sum(10,20,30);
}
public static void sum(int n1,int n2){
System.out.println("Sum of 2 integer numbers: "+(n1+n2));
}
public static void sum(int n1,int n2,int n3){
System.out.println("Sum of 3 integer numbers: "+(n1+n2+n3));
}
```

```
public static void sum(double n1,double n2){
System.out.println("Sum of 2 double numbers: "+(n1+n2));
}
}
```

Output->

```
Sum of 2 integer numbers: 30
Sum of 2 double numbers: 30.9
Sum of 3 integer numbers: 60
```

2.6 Abstract Classes

- This class does nothing but only specifies the member functions that can be used in the hierarchy.
- It can have abstract and non-abstract method
- It needs to be extended and its method implemented.
- It cannot be instantiated.

Program

```
abstract class A{
abstract void fun1();
void fun2(){
System.out.println("fun2 in A");
}
}
class B extends A{
void fun1(){
System.out.println("fun1 of A in B");
}}
class C extends A{
void fun1(){
System.out.println("fun1 of A in C");
}}
public class Main{
public static void main(String args[]){
B b=new B();
C c=new C();
b.fun1();
b.fun2();
c.fun1();
c.fun2();
}}
```

Output->

```
fun1 of A in B
fun2 in A
fun1 of A in C
fun2 in A
```

2.7 Generic Programming

Generic is a mechanism for creating a general model in which generic methods and generic classes enable programmers to specify a single method or a set of related methods and single class or a set of related classes for performing the desired task

Need for generic

- Using Generics, it is possible to create classes that work with different data types.
- It allows code reusability
- Compact code can be created

2.7.1 Generic methods

Generic methods allow programmers to write a generalized method for the methods of different data type.

Example

```
public class Demo{
public static<T extends Comparable<T>> void display(T[] a){
for(int i=0;i<5;i++)
System.out.print(a[i]+"  ");
}
public static void main(String args[]){
Float dbl_a[]={1.1f,2.2f,3.3f,4.4f,5.5f};
Integer int_a[]={1,2,3,4,5};
String str_a[]={"AA","BB","CC","DD","EE"};
System.out.print("Float elements are: ");
display(dbl_a);
System.out.print("\nInteger elements are: ");
display(int_a);
System.out.print("\nString elements are: ");
display(str_a);
}
}
```

Output

```
Float elements are: 1.1  2.2  3.3  4.4  5.5
Integer elements are: 1  2  3  4  5
String elements are: AA  BB  CC  DD  EE
```

2.7.2 Generic Classes

A generic class contains one or more generic data type

Example

```
public class Demo<T extends Comparable<T>>{
Demo(T[] a){
for(int i=0;i<5;i++)
System.out.print(a[i]+"  ");
}
public static void main(String args[]){
Float dbl_a[]={1.1f,2.2f,3.3f,4.4f,5.5f};
Integer int_a[]={1,2,3,4,5};
String str_a[]={"AA","BB","CC","DD","EE"};
System.out.print("Float elements are: ");
Demo<Float> dbl_D=new Demo<Float>(dbl_a);
System.out.print("\nInteger elements are: ");
Demo<Integer> int_D=new Demo<Integer>(int_a);
System.out.print("\nString elements are: ");
Demo<String> str_D=new Demo<String>(str_a);
}
}
```

Output

```
Float elements are: 1.1  2.2  3.3  4.4  5.5
Integer elements are: 1  2  3  4  5
String elements are: AA  BB  CC  DD  EE
```

2.7.3 Bounded Types

While creating objects to generic classes, we can pass any derived type as type parameters. Manytimes, it will be useful to limit the types thatcanbe passed to type parameters.for that purpose, bounded types are introduced in generics.

Using bounded types, we can make the objects of generic class to have data of specific derived type.

Example

```
public class Demo<T extends Number>{
Demo(T[] a){
for(int i=0;i<5;i++)
System.out.print(a[i]+"  ");
}
public static void main(String args[]){
Float dbl_a[]={1.1f,2.2f,3.3f,4.4f,5.5f};
Integer int_a[]={1,2,3,4,5};
String str_a[]={"AA","BB","CC","DD","EE"};
System.out.print("Float elements are: ");
Demo<Float> dbl_D=new Demo<Float>(dbl_a);
System.out.print("\nInteger elements are: ");
Demo<Integer> int_D=new Demo<Integer>(int_a);
//System.out.print("\nString elements are: ");
//Demo<String> str_D=new Demo<String>(str_a);
}}
```

Output

```
Float elements are: 1.1  2.2  3.3  4.4  5.5
Integer elements are: 1  2  3  4  5
```

2.7.4 Restrictions and limitations

1. Generic types are compile time entities. The runtime execution is possible only if it is used along with raw type.
2. Primitive type parameters are not allowed for generic programming.
3. For the instance of generic class, throw and catch instances are not allowed.
4. Instantiation of generic parameter T is not allowed
   Eg. new T(); new t[]; // Error
5. Arrays of parameterized types are not allowed
   Eg. new stack<String>[10]; // Error
6. Static fields and static methods with type parameters are not allowed

2.8 Inner Classes

- Inner classes are the nested classes.
- Classes defined inside other classes

Syntax

```
Access_modifier class outer_class{
// code
        Access_modifier class inner_class{
      // code
      }
}
```

2.8.1 Properties of inner class

- Outer class inherits as many number of inner class objects
- If both outer and inner classes are public, then any other classes can create an instance of the inner class
- Inner class objects do not get instantiated with outer class object
- Outer class can call even the private methods of inner class
- Inner class code has free access to all elements of the outer class object that contains it.
- If inner class has a variable with same name, then the outer class' variable can be accessed like this

```
Outer_class_name.this.variable_name;
```

2.8.2 Types of inner classes

1. Static member classes

- Inner class is a static class
- Static members of outer class are visible to static inner class
- Non Static members of outer class are not visible to static inner class

Syntax

```
Access_modifier class outer_class{
// code
  public static class inner_class{
  // code
    }
}
```

Program

```
class Outer {
public void showOuter()
{
System.out.println("In Outer Class Method");
}
static class Inner {
public static void showInner()
{
System.out.println("In Inner Class Method ");
}
}
}
class Main {
public static void main(String[] args)
{
Outer out=new Outer();
out.showOuter();
Outer.Inner.showInner();
}
}
```

Output->

```
In Outer Class Method
In Inner Class Method
```

2. Member classes

- Inner class is a non static class

Syntax

```
Access_modifier class outer_class{
// code
  Access_modifier class inner_class{
  // code
    }
}
```

Example

```
class Outer {
public void showOuter()
{
System.out.println("In Outer Class Method");
}
class Inner {
public void showInner()
{
System.out.println("In Inner Class Method ");
}
}
}
class Main {
public static void main(String[] args)
{
Outer out=new Outer();
Outer.Inner in=out.new Inner();
out.showOuter();
```

```
in.showInner();
}
}
```

Output->

```
In Outer Class Method
In Inner Class Method
```

3. Local Classes

- Never declared with access specifier
- Inner class can be accessed only within the block in which it is defined
- Hidden from the outer world

Syntax

```
Access_modifier class outer_class{
// code
  Access_modifier return_type method_name(arguments){
  class inner_class{
  // code
  }}
}
```

Example

```
class Outer {
public void showOuter()
{
System.out.println("In Outer Class Method");

class Inner {
void showInner()
{
System.out.println("In Inner Class Method ");
}
}
Inner in=new Inner();
in.showInner();
}
}
class Main {
public static void main(String[] args)
{
Outer out=new Outer();
out.showOuter();
}
}
```

Output->

```
In Outer Class Method
In Inner Class Method
```

4. Anonymous Classes

- Local class without any name
- One-shot class created where needed
- Created in following situations
    - When class has very short body
    - Only one instance of the class is needed
    - Class is used immediately after defining it

Example

```
interface Anonymous{
void fun();
}
class TestAnonymousInner{
```

```
public static void main(String args[]){
Anonymous A=new Anonymous(){
public void fun(){
System.out.println("Anonymous Class");
} };
A.fun();
} }
```

Output->

Anonymous Class

2.9 Binding

Binding is a mechanism creating link between method call and method actual implementation. As per the polymorphism concept in Java, object can have many different forms. Object forms can be resolved at compile time and run time.

2.9.1 Java Static Binding

Static binding refers to the process in which linking between method call and method implementation is resolved at compile time. Static binding is also known as compile-time binding or early binding.

```
class Calculator{
  public static int add(int a, int b){
    return a + b;
  }
  public static int add(int a, int b, int c){
    return a + b + c;
  }}
public class Tester {
  public static void main(String args[]){
    System.out.println(Calculator.add(20, 40));
    System.out.println(Calculator.add(40, 50, 60));
  }}
```

Output:

60

150

2.9.2 Dynamic Binding

Dynamic binding refers to the process in which linking between method call and method implementation is resolved at run time (or, a process of calling an overridden method at run time). Dynamic binding is also known as run-time polymorphism or late binding. Dynamic binding uses objects to resolve binding.

```
class A {
  public void fun() {
    System.out.println("In fun of class A");
  }}
class B extends A {
  public void fun() {
    System.out.println("In fun of class B");
  }}
public class Tester {
  public static void main(String args[]) {
    A a = new A();   // A reference and object
    A b = new B();   // A reference but B object (Dynamic Binding)
    a.fun();   // runs the method in Aclass
    b.fun();   // runs the method in B class
  }}
```

Output:

In fun of class A

In fun of class B

2.10 Interfaces

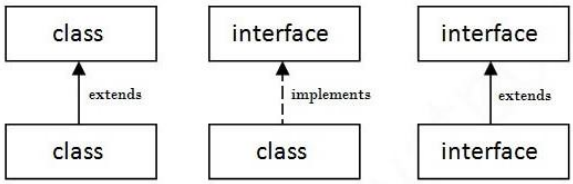
- An interface in Java is a blueprint of a class.
- It has static constants and abstract methods.
- mechanism to achieve abstraction.
- There can be only abstract methods in the Java interface, not method body.
- used to achieve abstraction and multiple inheritance in Java.

2.10.1 Defining interface

Syntax:

```
interface <interface_name>{
   // declare constant fields
   // declare methods that abstract by default.
}
```

As shown in the figure given below, a class extends another class, an interface extends another interface, but a class implements an interface.



2.10.2 Implementing interface

```
interface sample_interface{
void print();
}
```

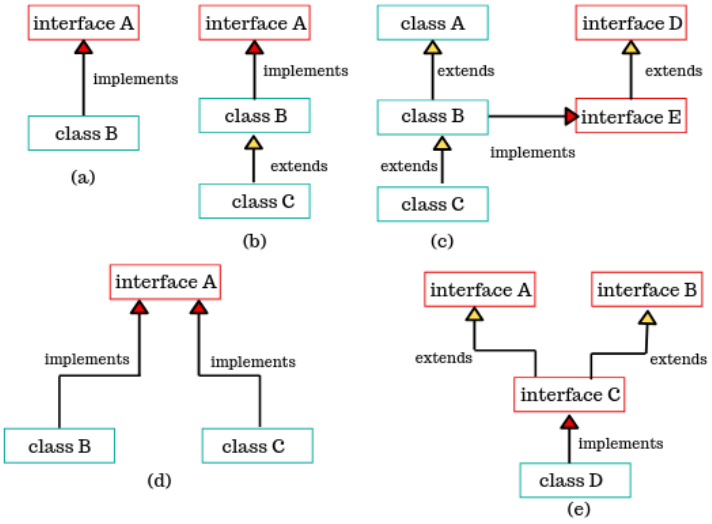
Save it as sample_interface.java, but don't compile it. Create another file named Demo that implements the above interface.

```
class Demo implements sample_interface{
public void print(){
System.out.println("Hello");
}
public static void main(String args[]){
Demo obj = new Demo();
obj.print();
}
}
```

Output

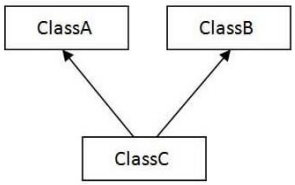
Hello

Various forms of interface implementation



Multiple Inheritance

A single class is derived from more than one base class



Java does not support multiple inheritance. But it implements the concept of multiple inheritance through interfaces.

```
interface A{
   public void showA();
}
interface B{
   public void showB();
}
class C implements A,B{
public void showA(){
System.out.println("From interface A");
}
public void showB(){
System.out.println("From interface B");
}
public static void main(String args[]){
  C obj=new C();
obj.showA();
obj.showB();
}}
```

Output:

From interface A
From interface B

2.10.3 Extending interfaces

We can extend sub interfaces from main interfaces using ‘extends’ keyword

```
interface A{
int val=10;
}
interface B extends A{
void print();
}
class C implements B{
public void print(){
System.out.println(val);
}
public static void main(String args[]){
C c=new C();
c.print();
}}
```

Output

10

2.11 Packages

- A java package is a group of similar types of classes, interfaces and sub-packages.

2.11.1 Package as Access Protection

Packages help avoid name conflicts and control access.

Java provides four access levels:

Modifier	Class	Package	Subclass	World
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

- Same package classes can access each other's default, protected, and public members.
- Classes outside the package conly access public members.

2.11.2 CLASSPATH Setting

Package names correspond to a directory structure. Java compiler and runtime must be informed about the location of the base directory of the package so as to locate the classes.
It is is accomplished by an environment variable called CLASSPATH.

- **Temporary (Command Line)**:

```
java -cp .;C:\MyClasses MyApp   # Windows
```

- **Permanent (Environment Variable)**:

```
set CLASSPATH=.;C:\MyClasses
```

2.11.3 Types of Packages in Java

Based on whether the package is defined by the user or not, packages are divided into two categories:

1. Built-in Packages
2. User Defined Packages

1. Built-in Packages

The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.The library is divided into packages and classes. Meaning you can either import a single class, or a whole package that contain all the classes that belong to the specified package. To use a class or a package from the library, you need to use the **import** keyword:

Syntax

```
import package.name.Class;   // Import a single class
import package.name.*;   // Import the whole package
```

Example

```
import java.util.Scanner;
class MyClass
{
  public static void main(String[] args)
{
    Scanner myObj = new Scanner(System.in);
System.out.println("Enter username");
    String userName = myObj.nextLine();
System.out.println("Username is: " + userName);
  }
}
```

Output

```
Enter username
JACSICE
Username is: JACSICE
```

2. User-defined Packages

To create your own package, you need to understand that Java uses a file system directory to store them. Just like folders on your computer:

```
Example
└── root
  └── mypack
    └── MyPackageClass.java
```

To create a package, use the package keyword:

A.java

```
package pack;
public class A{
public void msg1(){System.out.println("Hello");}
}
```

Save the file as A.java, and compile it:

javac A.java

B.java

```
package pack;
public class B{
public void msg2(){System.out.println("How are you?");}
}
```

Save the file as B.java, and compile it:

javacB.java

C.java

```
import pack.*;
class C{
  public static void main(String args[]){
    A obj = new A();
    obj.msg1();
    B obj = new B();
    obj.msg2();

  }
}
```

Save the file as C.java, and compile it:

Output:

```
Hello
How are you?
```